



230968336

SAMEER RAJ

WEEK-4

QUES-1

```
In [1]: import heapq

def manhattan_distance(node, goal):
    return abs(node[0] - goal[0]) + abs(node[1] - goal[1])

def greedy_best_first_search(maze, start, goal):
    """
    Implements Greedy Best-First Search for maze navigation
    """

    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    frontier = []
    heapq.heappush(frontier, (manhattan_distance(start, goal), start))

    visited = set()
    visited.add(start)

    parent = {start: None}

    nodes_expanded = 0

    while frontier:
        _, current = heapq.heappop(frontier)
        nodes_expanded += 1

        if current == goal:
            break

        for move in directions:
            next_node = (current[0] + move[0], current[1] + move[1])
```

```

        if (0 <= next_node[0] < len(maze) and
            0 <= next_node[1] < len(maze[0]) and
            maze[next_node[0]][next_node[1]] == 0 and
            next_node not in visited):

            visited.add(next_node)
            parent[next_node] = current
            h = manhattan_distance(next_node, goal)
            heapq.heappush(frontier, (h, next_node))

    path = []
    node = goal
    while node is not None:
        path.append(node)
        node = parent.get(node)

    path.reverse()

    if path[0] == start:
        return path, nodes_expanded
    else:
        return None, nodes_expanded

maze = [
    [0, 0, 0, 1, 0],
    [1, 1, 0, 1, 0],
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 0, 0]
]

start = (0, 0)
goal = (4, 4)

path, nodes_expanded = greedy_best_first_search(maze, start, goal)

if path:
    print("Path found:")
    print(path)
else:
    print("No path found.")

print("Number of nodes expanded:", nodes_expanded)

```

Path found:

[(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (2, 4), (3, 4), (4, 4)]

Number of nodes expanded: 9

QUES-2

```
In [2]: import heapq

def manhattan_distance(node, goal):
    return abs(node[0] - goal[0]) + abs(node[1] - goal[1])

def a_star_search(maze, start, goal):
    """
    Implements A* Search algorithm for optimal maze navigation
    """

    directions = [(-1, 0), (1, 0), (0, -1), (0, 1)]

    frontier = []
    heapq.heappush(frontier, (0, 0, start))

    g_cost = {start: 0}

    parent = {start: None}

    nodes_expanded = 0

    while frontier:
        f, g, current = heapq.heappop(frontier)
        nodes_expanded += 1

        if current == goal:
            break

        for move in directions:
            next_node = (current[0] + move[0], current[1] + move[1])

            if (0 <= next_node[0] < len(maze) and
                0 <= next_node[1] < len(maze[0]) and
                maze[next_node[0]][next_node[1]] == 0):

                new_g = g + 1 # uniform cost

                if next_node not in g_cost or new_g < g_cost[next_node]:
                    g_cost[next_node] = new_g
                    h = manhattan_distance(next_node, goal)
                    f = new_g + h
```

```

        heapq.heappush(frontier, (f, new_g, next_node))
        parent[next_node] = current

    path = []
    node = goal
    while node is not None:
        path.append(node)
        node = parent.get(node)

    path.reverse()

    if path[0] == start:
        return path, nodes_expanded
    else:
        return None, nodes_expanded

maze = [
    [0, 0, 0, 1, 0],
    [1, 1, 0, 1, 0],
    [0, 0, 0, 0, 0],
    [0, 1, 1, 1, 0],
    [0, 0, 0, 0, 0]
]

start = (0, 0)
goal = (4, 4)

path, nodes_expanded = a_star_search(maze, start, goal)

if path:
    print("Optimal path found by A* Search:")
    print(path)
else:
    print("No path found.")

print("Number of nodes expanded:", nodes_expanded)

```

Optimal path found by A* Search:

[(0, 0), (0, 1), (0, 2), (1, 2), (2, 2), (2, 3), (2, 4), (3, 4), (4, 4)]

Number of nodes expanded: 9

For both the experiments do the following:

Compare BFS, Greedy Best-First Search, and A* in terms of:

- Path length found
- Nodes expanded
- Optimality of solution
- Efficiency

Breadth-First Search (BFS): BFS always finds the shortest path in an unweighted maze. However, it expands a large number of nodes before reaching the goal. The solution is optimal, but the algorithm is inefficient due to high time and memory usage.

Greedy Best-First Search: Greedy Best-First Search may find a longer, sub-optimal path since it relies only on heuristic information. It expands the fewest nodes among the three algorithms. The solution is not guaranteed to be optimal, but the algorithm is highly efficient and fast.

A* Search: A* Search always finds the shortest path when using an admissible heuristic. It expands fewer nodes than BFS but more than Greedy Best-First Search. The solution is optimal, and the algorithm is efficient, offering a good balance between speed and accuracy.